

NovaCart Order Analytics: Medallion Pipeline Cheatsheet

Team Bug Byts · HCLTech AI-Ready Cloud Data Engineer lab · status as of 6 Oct 2026 · repo: github.com/sayantansikdar/novacart-medallion-pipeline

The task. Build a bronze → silver → gold pipeline for NovaCart's September 2026 orders that gives the right answer after batch 1 (1–15 Sep), after batch 2 (new orders + corrected versions of earlier ones), and when either batch is accidentally run twice. Inputs may never be edited by hand; every assumption goes in the design note; AI use must be disclosed and every line explainable in a 30-minute viva.

1 · Platform: what we use and why

Step	Outcome	Lab requirement	Fabric feature
Azure for Students	Subscription disabled (offer ended 19 Sep 2026); cannot be re-enabled from the CLI.	Raw + table storage	OneLake lakehouses (Bronze / Silver / Gold)
Databricks Free Edition	Set up and working, but dropped in favour of the team's Fabric workspace.	Upserts, history, point-in-time, schema enforcement	Delta tables: MERGE, DESCRIBE HISTORY, VERSION AS OF, NOT NULL + CHECK
Azure free trial	Enabled, but only Contributor rights (cannot grant the storage role the Databricks design needs).	Distributed processing	Fabric Spark notebooks (PySpark + Spark SQL)
Microsoft Fabric (chosen)	Team workspace NovaCart_HCL on the existing F2 capacity <i>hackafabric</i> , Central India. Resumed only while working, paused right after.	Parameter, retries, on-failure path	Fabric Data Pipeline (Module 7)
		Security	Entra ID sign-in only: no keys, tokens or secrets anywhere

Why Fabric: one platform covers every requirement natively; the tables are Delta, so the Spark/Delta code is the same as the original Databricks design; Power BI sits on top for the JD's BI expectation; and the F2 costs about **\$0.36/hour only while resumed** (≈ \$2 so far).

2 · Architecture and run order

Layer	Lakehouse / notebook	What happens
Input	LH_NovaCart_Bronze / Files/NovaCart_SourceData/...	8 raw files, uploaded once, SHA-256 identical to the originals. 3 file names differ from the brief: mapped in config, never renamed.
Setup	00_config, 00_setup	All names/paths/helpers in one place. Creates 8 silver tables with NOT NULL + 10 CHECK constraints, Change Data Feed on orders, the -1 unknown customer.
Bronze	01_bronze → 6 tables	Every column as STRING + _source_file, _batch_id, _ingested_at. Malformed lines kept in _corrupt_record. Write replaces only its own _source_file slice.
Silver	03_silver_reference → 02_silver_orders → 04_silver_items	Typed, cleaned, UTC. Bad rows → silver.quarantine (reason + original record). Counts → silver.dq_report. MERGE / snapshot sync, never blind overwrite.
Gold	05_gold	fact_order_line + daily_revenue, revenue_by_category, top_customers, payment_mismatches, reconciliation; DQ report exported as JSON.
Orchestration	Pipeline novacart_medallion	batch_id → setup → bronze → silver ×3 → gold; 2 retries per step; on failure: notify_failure (alert table + file) → fail_run.

3 · What profiling the data revealed (and how the design handles it)

Finding in the real data	Handled by
10 late batch-2 rows are older paid copies of orders batch 1 already marked cancelled	MERGE guard: only a newer updated_at may replace a row
236 +05:30 timestamps fall on the previous day in UTC	Session time zone UTC; order_date = UTC date
435 dd/MM/yyyy rows with day ≤ 12 (a month-first parser would silently be wrong)	Explicit dd/MM/yyyy pattern
49 batch-2 returns belong to batch-1 orders	Returns dated by the parent order; gold rebuilt each run
Orphan items NC-990xxx (10 + 8); unknown products P101–P105 (14 lines)	ORDER_NOT_FOUND quarantine; product_known = false
5 CRM rows (all Platinum) without updated_at; customers C0901–C0906 not in the CRM	Quarantined; customer_sk = -1
FX has no weekends and no Mon 7 Sep; 5 payments for unknown orders NC-80000x	As-of earlier rate; kept, excluded from mismatch check
Duplicates: 18 order rows and 25 item lines per batch; 12 item lines re-delivered	Deterministic dedup; MERGE skips identical rows

4 · Viva one-liners

Term	Say it like this
Medallion	Bronze = as received, silver = clean and conformed, gold = business-ready; each layer can be rebuilt from the one before.
Idempotent	Running the same batch twice leaves the tables exactly as after one run (proved: 0 inserted, 0 updated, no new version).
MERGE guard	Upsert that only accepts a newer updated_at, so late data cannot overwrite current data.
SCD Type 2	Keeps every tier/country version with valid_from/valid_to, so an order joins the customer as they were on the order date.
Time travel	Delta keeps old versions: VERSION AS OF reads the table as it was after batch 1. VACUUM deletes old files and limits this.
Quarantine	Bad rows are never dropped silently: each is stored with a reason and its original record, and counted in dq_report.

5 · Modules: what was built and the test results

#	Module	Key design	Result
0	Discovery	Profiled all 8 files locally; platform chosen	Data traps listed above
1	Foundation	Silver + Gold lakehouses (idempotent script); inputs verified	✓ SHA-256 identical; 8/8 files readable, counts match
2	Config + setup	Timestamp parser (3 formats), currency rule, quarantine + DQ helpers, DDL	✓ All pass : S1, S2, S3–S5, O1, R4, D1; second setup run = 0 commits
3	Bronze	STRING + lineage; replaceWhere on _source_file	✓ all pass (one test bug fixed and re-verified): B1–B6, D1; re-run counts unchanged
4	Silver orders	Parse → quarantine → latest per order → guarded MERGE	✓ all pass (R1 expectation corrected and re-verified): S3–S12, O4 ; 945 orders; status mix = profile exactly
5	Items, SCD2, reference	Items MERGE on (order_id, line_no); SCD2 on tier/country; snapshot sync	✓ all 38 pass (after fixing <i>try_to_date</i> , missing in Fabric's Spark 3.5): I1–I7, C1–C4, D1, O4 for items; re-run of batch 2 = no new commit on any table
6	Gold	As-of FX, point-in-time customer, revenue flag; full deterministic rebuild; reconciliation + DQ JSON export	✓ all 29 pass : G1–G9, C5, C6, I5; every gold total equals an independent plain-Python calculation to the cent
7	Pipeline	Fabric Data Pipeline <i>novacart_medallion</i> : batch_id parameter, 2 retries / 60 s, on-failure notify_failure → fail_run	✓ failure path (batch_id = abc: setup failed 3×, rest skipped, alert recorded, run Failed). ✓ Official runs after a full reset: batch 1, 2 and 2 again, every step Succeeded. ✓ 25/25 verification : R1, R2 (time travel), S10, gold after batch 1 and final = Python reference, D1 for both batches

6 · Key design decisions (the “why”)

- **Bronze keeps everything as text**: nothing is lost or silently converted; silver decides.
- **Re-run safety everywhere**: bronze replaces its own file slice; silver MERGE changes only what differs; quarantine and dq_report replace their own (entity, batch) rows. A second run = no change.
- **Late data**: update only if s.updated_at > t.updated_at. Same timestamp: later batch wins only if content differs, so **batch 2 then 1 = batch 1 then 2** (O4).
- **SCD Type 2** on tier and country; name/e-mail are Type 1; first version from 1900-01-01; customer_sk = hash(customer_id, valid_from), so keys are stable across rebuilds.
- **Money is DECIMAL**, never float; **all time is UTC**.
- **Every rejected row** is in silver.quarantine with a reason and its original record; dq_report enforces rows in = out + quarantined + deduped before writing.

7 · Real-data results so far

- silver.orders: batch 1 = 1418 rows in → 495 orders; batch 2 = 450 new, 137 updated → **945 orders**.
- Final status: delivered 666 · cancelled 107 · shipped 89 · placed 44 · paid 39 (the late copies were ignored).
- 48 orders have a currency derived from the shipping country; 0 orders quarantined.
- Re-run of batch 2: **0 inserted, 0 updated, and no new table version**: Delta writes no commit for a MERGE that changes nothing (official history: v3 = batch 1, v4 = batch 2, latest = v4).
- Time travel: silver.orders VERSION AS OF 3 = 495 orders with exactly the batch-1 status mix; gold read “as of batch 1” = the batch-1 Python reference (\$150,062.64, 64 mismatches).
- Bronze: 1418 / 1006 / 194 / 40 / 84 / 1164 rows per file, cross-checked by an independent count.
- Items: **1933 lines** (971 + 974 - 12 re-delivered); 10 + 8 orphans quarantined; 14 unknown-product lines flagged.
- dim_customer: **166 versions** for 130 customers (+ the -1 row); 5 CRM rows quarantined; no overlaps or gaps.
- Gold (final): revenue **\$295,585.57** over 30 days (Electronics \$223,943.51); top customer C0035; **13** payment mismatches; 5 payments for unknown orders. After batch 1 only: \$150,062.64 over 15 days.

8 · Fabric lessons learned

- A child notebook started from another must share its default lakehouse.
- notebookutils.fs.head returns at most ~100 KB: read whole files with Spark instead.
- Every failed run reports the same generic error, so tests save PASS/FAIL details to a JSON file.

9 · Remaining stages

Module	Plan
7 Screens	Screenshots: app.fabric.microsoft.com → Monitor → novacart_medallion run → View run details (batch 1, batch 2, abc failure). Trigger design in the note: file-arrival event vs fixed or windowed schedule.
8 Evidence	History after batch 1 / 2 / 2 again; VERSION AS OF right after batch 1; schema enforcement (extra column rejected) + CHECK violation; OPTIMIZE / V-Order before vs after; VACUUM warning (limits time travel). Extras: Change Data Feed rows changed in batch 2, incremental gold, automatic ingestion.
9 Submission	Test summary, submission checklist, AI-use disclosure per module, design note (≤ 1.5 pages), teardown (pause or delete the F2, clean the workspace).

10 · Design-note updates needed

- Tools: **Fabric Lakehouse (OneLake + Delta)**, **Fabric Spark**, **Data Pipelines** replace ADLS Gen2, Databricks, Workflows and Unity Catalog.
- History diagram: v3 = batch 1, v4 = batch 2, **no version for the batch 2 re-run** (not “v4: 0 changes”).
- Assumptions to add: payments for unknown orders are kept and excluded from mismatches; same-updated_at tie goes to the later batch only if content differs; if batch 2 runs before batch 1, its lines for batch-1 orders stay quarantined (auto-retry = future work).
- Payment mismatch: expected = order net total if paid/shipped/delivered, else 0 (a cancelled order must be fully refunded). The payments file is a full-month snapshot, so after batch 1 it already holds batch-2 payments (64 mismatches then, 13 at the end).